Lecture 4: Algorithmic Complexity

Growth rates, asymptotic notation, and what they do and do not tell us

Handout 4 of 11 · Department of Computer Science
Lecture: Tuesday, 11:00 · Lab session: Thursday, 14:00
Prerequisite: Lecture 3 (arrays, lists, recursion)
Assessed by: Coursework 1, question 2

These notes accompany Lecture 4 and are written to be read on their own, so the worked examples are given in full rather than left on the whiteboard. Everything here is examinable, and question 2 of Coursework 1 assumes you have attempted the exercises at the end.

Learning outcomes

- State the definition of Big-O notation and apply it to a polynomial cost function.
- Order the common complexity classes by growth rate, and distinguish best-, average- and worst-case analysis.
- Explain why an asymptotic classification does not by itself determine which of two algorithms runs faster on a given input.
- Distinguish auxiliary space from total space, and classify a sort as in-place or out-of-place.

1. Why we measure complexity

Two students submit programs that solve the same problem. On the same test file, one finishes in 0.8 seconds and the other in 2.1 seconds, and it is tempting to conclude that the first student has the better algorithm. The measurement does not support that. What was measured is the behaviour of one implementation, written in one language, compiled with one set of options, running on one machine, against one input. Change any of those and the ranking can change.

The confounding factors are numerous. Processor speed and instruction set matter, and so does the memory hierarchy: an algorithm whose working set fits in cache can beat one that does fewer operations but scatters them across memory. Choice of language matters enormously, since a careless program in a compiled language routinely outruns a careful one in an interpreted language by a factor of thirty or more, which is easily enough to disguise a genuinely worse method.

We want to compare the methods, not the circumstances, so the standard approach abstracts away from the machine entirely. Choose a measure of input size, normally written n: the number of elements in an array, the number of vertices and edges in a graph. Choose an elementary operation that dominates the work, such as a comparison in a sorting routine. Then count how many times that operation is performed as a function of n, and call the result T(n). Because T(n) counts operations rather than seconds, it is the same whoever compiles the program and whatever hardware they run it on. It is also predictive in a way a stopwatch is not: a timing tells you about the input you happened to try, whereas a cost function tells you what will happen when the input is a thousand times larger. Systems are rarely retired for being slow on today's data; they are retired because the data grew and the cost grew faster.

A small example shows what this buys us. Suppose algorithm A performs $100n$ elementary operations and algorithm B performs n^2. For $n = 10$, A does 1,000 operations and B does 100, so B is ten times cheaper. At $n = 100$ they are level, at 10,000 each. For $n = 1{,}000$, A does 100,000 and B does 1,000,000, so A is now ten times cheaper and the gap keeps widening. Neither algorithm is simply "the fast one": which is faster depends on n, and the two curves cross at a specific place. We return to that crossing point in Section 5.

2. Big-O notation

Counting exact operations is still more detail than we want. The counts $3n^2 + 5n + 100$ and $7n^2 + 2n + 4$ describe algorithms that behave the same way as n grows, even though the numbers differ. Big-O notation expresses that shared behaviour and discards the rest.

> **Definition.** Let f and g be functions from the positive integers to the non-negative reals. We say that *f(n) is O(g(n))* if there exist a constant $c > 0$ and an integer $n_0 \geq 1$ such that $f(n) \leq c \cdot g(n)$ for every $n \geq n_0$.

In words: from some point onwards, f is bounded above by a constant multiple of g. Three features of that definition deserve to be read carefully, because each of them limits what a Big-O statement is entitled to claim.

It is an upper bound, not an exact description. The definition says f grows *no faster than* g; it does not say f grows as fast as g. A function that is O(n) is also, correctly, $O(n^2)$ and $O(2^n)$, because a loose bound is still a bound. By convention we quote the tightest bound we can establish, which is why saying "binary search is $O(n^2)$" would be true but useless.

It is asymptotic. The definition constrains f only for $n \geq n_0$, and it does not say how large n_0 is. Below that threshold the inequality is allowed to fail completely, and the notation makes no claim of any kind about what happens there. The constant c is similarly unconstrained; it may be 2, or it may be two million. Section 5 is about the consequences of these two silences.

It describes growth rate, not speed. This is the point most often lost. Big-O measures growth rate, not speed: it tells you how the running time scales as the input size increases, not how many seconds the algorithm takes on any particular machine. Two algorithms in the same Big-O class can differ in real running time by a factor of a hundred, because the constant factor that separates them is exactly what the notation throws away. Because the constant factor is discarded, an algorithm with a better Big-O classification is not guaranteed to be faster than one with a worse classification on any particular input; the classification only tells you which one must eventually win as n grows without bound.

Dropping constants and lower-order terms

In practice we do not work from the definition every time. We simplify a cost function by two rules: discard all constant multipliers, and discard every term except the fastest-growing one. So $3n^2 + 5n + 100$ is $O(n^2)$. The 3 goes because constant multipliers are dropped; the 5n and the 100 go because they are lower-order terms, dominated by n^2.

The justification is arithmetic. As n grows, the quadratic term takes over completely:

n	$3n^2$	5n	100	total T(n)	share from $3n^2$
10	300	50	100	450	66.7%
100	30,000	500	100	30,600	98.0%
1,000	3,000,000	5,000	100	3,005,100	99.8%

Table 1. Contribution of each term of $T(n) = 3n^2 + 5n + 100$. By n = 1,000 the linear and constant terms together account for less than a fifth of one percent of the total.

To confirm it formally we need only exhibit a c and an n_0. For every $n \geq 1$ we have $5n \leq 5n^2$ and $100 \leq 100n^2$, so $3n^2 + 5n + 100 \leq 3n^2 + 5n^2 + 100n^2 = 108n^2$. Taking $c = 108$ and $n_0 = 1$ satisfies the definition, and therefore T(n) is $O(n^2)$. A tighter pair also works: for $n \geq 10$, $T(n) \leq 4.5n^2$, with $c = 4.5$ and $n_0 = 10$. Both are valid, because the definition asks for the existence of some c and n_0 rather than the best ones.

Be explicit about why discarding the constants is legitimate, because the same step is what makes the result blunt. The constant multiplier reflects how much a single elementary step costs on a given machine with a given compiler, precisely the factors Section 1 set out to eliminate; removing them is what makes the classification portable. The price is that it then retains no information about the regime in which those constants dominate. Big-O answers one question, how cost scales with input size, and it answers no other question.

Two companions appear in the literature: $\Omega(g(n))$ is the mirror image, an asymptotic *lower* bound, and $\Theta(g(n))$ is an asymptotically tight bound, meaning both at once.

3. The growth-rate hierarchy

A small number of complexity classes account for most of the algorithms you will meet. Learning their relative behaviour lets you judge quickly whether a proposed approach is plausible at the scale you care about.

Class	Name	Typical example	n = 10	n = 100	n = 1,000	n = 1,000,000
O(1)	constant	array index; hash lookup	1	1	1	1
O(log n)	logarithmic	binary search on sorted data	3	7	10	20
O(n)	linear	linear search; one pass over a list	10	100	1,000	10^6
O(n log n)	linearithmic	mergesort; heapsort; quicksort (average)	33	664	9,966	2.0×10^7
O(n²)	quadratic	insertion sort; comparing all pairs	100	10,000	10^6	10^{12}
O(2ⁿ)	exponential	enumerating all subsets	1,024	1.3×10^{30}	$\approx 10^{301}$	beyond astronomical

Table 2. Approximate operation counts by complexity class. Logarithms are base 2 and figures are rounded.

A useful way to internalise the table is to ask what happens when the input doubles. An O(1) algorithm is unaffected and an O(log n) algorithm performs one extra step. An O(n) algorithm does twice the work, an O(n log n) algorithm slightly more than twice, and an O(n²) algorithm four times, so a tenfold increase in input becomes a hundredfold increase in cost. An O(2^n) algorithm squares its own workload, which is why exponential algorithms hit a wall hardware cannot move: a machine a thousand times faster buys roughly ten more elements of input.

Note also that the base of the logarithm never appears in the classification, since changing base multiplies by a constant. And the classes are not equally spaced in practice: the step from O(n²) to O(n log n) usually decides whether a program is viable on large data, whereas the step from O(n) to O(n log n) is often invisible, since at n = 1,000,000 the factor $\log_2 n$ is only about 20.

4. Best case, average case and worst case

Input size alone does not determine how much work an algorithm does: two arrays of the same length can send the same routine down very different paths. A complete analysis therefore reports three quantities, the fewest operations over all inputs of size n (best case), the most (worst case), and the expected number under a stated distribution (average case). These are separate results, frequently in different complexity classes.

Linear search

```
function LINEAR-SEARCH(A, target):
    for i = 0 to length(A) - 1:
        if A[i] == target:
            return i                # found it; stop immediately
    return NOT_FOUND                # fell off the end
```

The best case is one comparison, when the target sits at index 0, so O(1). The worst case is n comparisons, when the target is in the last position or absent altogether, so O(n). For the average case we must state an assumption: if the target is present and equally likely to be at any index, the expected number of comparisons is (n + 1) / 2, still O(n) once the constant is dropped. An average-case result is only as trustworthy as the assumption behind it.

Quicksort

```
function QUICKSORT(A, lo, hi):
    if lo >= hi:
        return                      # zero or one element: already sorted
    p = PARTITION(A, lo, hi)        # choose a pivot, move smaller items left
    QUICKSORT(A, lo, p - 1)         # sort the left part
    QUICKSORT(A, p + 1, hi)         # sort the right part
```

Each call to PARTITION examines every element between lo and hi once, so the work at any one level of the recursion is at most n. The total cost is therefore n times the depth of the recursion, and everything depends on how well the pivot splits the array.

When pivots land near the median, each call halves the range and the recursion is about $\log_2 n$ levels deep. That gives n × log n work overall, so quicksort is O(n log n) in the average case, and random inputs are overwhelmingly likely to behave this way. When pivots land badly the picture changes completely. If the pivot is always the smallest or largest remaining element, each call removes just one item and the recursion is n levels deep, giving n(n + 1) / 2 comparisons. So quicksort is O(n log n) on average but $O(n^2)$ in the worst case. The classic way to trigger the worst case is to hand a last-element-pivot implementation an array that is already sorted, an unfortunately common situation in real systems.

Practical implementations defend against this rather than accepting it. Choosing the pivot at random, or as the median of the first, middle and last elements, makes the degenerate case vanishingly unlikely to arise by accident. Introsort goes further and monitors recursion depth, switching to heapsort if the depth exceeds roughly $2 \log_2 n$, which converts the $O(n^2)$ worst case into a hard O(n log n) guarantee while keeping quicksort's speed on typical input.

Which case you should care about is an engineering question. A batch job that runs overnight cares about the average; a request handler with a hard latency budget cares about the worst case, because an average is no defence when a deadline is missed.

5. Why asymptotic analysis can mislead on small inputs

Return to the definition in Section 2. It guarantees a bound only for $n \geq n_0$, and it allows the constant c to be arbitrarily large. Both of those allowances have practical consequences, and together they mean that a Big-O classification tells you nothing whatsoever about small inputs.

The constants that were dropped are not fictions. They stand for real work: the overhead of a function call, pushing and popping a stack frame, pivot selection, bounds checks, allocating a scratch buffer, and above all the memory access pattern, since a sequential sweep through a small array may sit entirely in L1 cache while a cleverer algorithm jumps around and misses. On a large input these costs are diluted across a great deal of productive work; on a small input they are the whole cost.

Sorting is the standard illustration. Insertion sort is $O(n^2)$, which sounds disqualifying, but its constant factor is about as small as an algorithm's can be. Each step performs one comparison and one shift, it touches memory sequentially, it makes no recursive calls, and it allocates nothing. It also has an O(n) best case, so on nearly ordered data it barely does any work. Quicksort's per-call overhead is small but not zero, and on a twelve-element array you pay it repeatedly in exchange for very little actual sorting.

```
function INSERTION-SORT(A, lo, hi):
    for i = lo + 1 to hi:
        key = A[i]
        j = i - 1
        while j >= lo and A[j] > key:
            A[j + 1] = A[j]          # shift the larger element right
            j = j - 1
        A[j + 1] = key
```

An illustrative cost model makes the shape of the trade-off visible. Suppose insertion sort costs about $0.5n^2$ units and quicksort about $2n \log_2 n + 30$ units, the trailing 30 standing for per-call setup that does

not shrink with n:

n	insertion sort ≈ $0.5n^2$	quicksort ≈ $2n \log_2 n + 30$	faster in this model
8	32	78	insertion sort, by about 2.4×
16	128	158	insertion sort, narrowly
20	200	203	level pegging (the crossover)
32	512	350	quicksort
100	5,000	1,359	quicksort, by about 3.7×
1,000	500,000	19,962	quicksort, by about 25×

Table 3. An illustrative model, not a measurement; the exact constants depend on the machine and the implementation. The shape, however, is real, and so is the location of the crossover.

Below the crossover the algorithm with the worse asymptotic complexity is genuinely the faster one, on the same data and the same machine. Above it the asymptotically better algorithm pulls ahead and never gives the lead back. Both halves of that sentence are ordinary consequences of the definition; neither is an anomaly.

> **In practice.** This is not a theoretical curiosity: production sorting routines are built around it. Introsort, the hybrid used by the C++ standard library's std::sort, and Timsort, used by Python's list.sort and by Java's sort for object arrays, both stop subdividing once a partition or run falls below a fixed threshold of roughly 10 to 30 elements and finish that block with insertion sort, precisely because insertion sort's lower overhead beats quicksort's asymptotically better complexity on small arrays. The exact cut-off is a tuning constant, 16 in one widely used std::sort implementation and a few dozen in Timsort, and library authors treat choosing it as routine engineering rather than as a special case.

The hybrid is easy to write, and you will implement one in the lab session:

```
SMALL = 16                          # tuned per library; typically 10 to 30

function HYBRID-SORT(A, lo, hi):
    if hi - lo + 1 <= SMALL:
        INSERTION-SORT(A, lo, hi)   # cheaper here, despite being O(n^2)
        return
    p = PARTITION(A, lo, hi)
    HYBRID-SORT(A, lo, p - 1)
    HYBRID-SORT(A, p + 1, hi)
```

The same pattern appears outside sorting: implementations of Strassen's matrix multiplication only switch to it above dimensions in the hundreds, and a linear scan of a twenty-element array frequently beats a hash table lookup. The guidance follows directly. Use asymptotic analysis to choose the family of algorithms when inputs are large or unbounded, but not to choose between candidates when the input is small or tightly bounded; there, measure and expect the constants to decide.

6. Space complexity

Memory is analysed the same way and with the same notation: space complexity expresses how the memory an algorithm requires grows with input size n. One distinction must be kept clear, between two things people mean by "the space it uses".

Total space counts everything, including the input itself. Since any algorithm that reads all of its input occupies at least O(n) total space, this measure rarely distinguishes one algorithm from another and is not usually what is quoted. **Auxiliary space** counts only the extra memory allocated beyond the input: temporary arrays, bookkeeping structures, and the recursion stack. That is the interesting figure, and it is what is meant when a textbook says an algorithm "uses O(1) space". An algorithm whose auxiliary space is O(1), or O(log n) for the recursion stack alone, is called **in-place**; one that needs auxiliary

space proportional to the input is **out-of-place**.

Algorithm	Time (average)	Time (worst)	Auxiliary space	In-place?
Insertion sort	$O(n^2)$	$O(n^2)$	O(1)	yes
Heapsort	O(n log n)	O(n log n)	O(1)	yes
Quicksort	O(n log n)	$O(n^2)$	O(log n) stack, O(n) worst	yes
Mergesort	O(n log n)	O(n log n)	O(n)	no
Timsort	O(n log n)	O(n log n)	O(n)	no

Table 4. Time and auxiliary space for the sorting algorithms discussed in this module.

Two rows deserve comment. Mergesort is the canonical out-of-place sort: merging two sorted halves requires somewhere to put the result, so it needs a scratch buffer proportional to n, and in exchange it offers an O(n log n) guarantee in the worst case. Quicksort is called in-place because it rearranges elements within the original array, but that is not free: each pending recursive call occupies a stack frame, so balanced partitions cost O(log n) auxiliary space and degenerate ones O(n). Forgetting that the recursion stack is auxiliary space is a common mistake.

Time and space are frequently exchangeable: memoising a recursive function trades O(n) memory for an often dramatic reduction in time. The trade is not always available on favourable terms, though. When the dataset only just fits in memory, auxiliary space is the binding constraint and an out-of-place algorithm is not an option however good its time bound looks.

Summary

- Timing measurements describe an implementation on a machine; complexity analysis describes the method. f(n) is O(g(n)) when $f(n) \le c \cdot g(n)$ for all n beyond some n_0, an asymptotic upper bound on growth rate.
- Big-O describes how cost scales with input size, not how many seconds an algorithm takes. Constant factors and lower-order terms are dropped, so $3n^2 + 5n + 100$ is $O(n^2)$.
- Best, average and worst case are separate results. Quicksort is O(n log n) on average and $O(n^2)$ in the worst case.
- Because the definition says nothing about inputs below n_0, an asymptotically worse algorithm can be the faster choice on small inputs, which is why standard libraries fall back to insertion sort below about 10 to 30 elements.
- Auxiliary space excludes the input and includes the recursion stack; it is what "in-place" refers to.

Exercises (not assessed; discussed in Thursday's lab)

1. Show from the definition that $7n + 2n \log_2 n + 40$ is O(n log n), by exhibiting a suitable c and n_0.
2. An algorithm performs $T(n) = 6n^2 + 200n$ operations. For which n does the quadratic term exceed the linear term, and what does that tell you about when the label $O(n^2)$ becomes informative?
3. A team replaces an $O(n^2)$ routine with an O(n log n) one and the program gets slower. Give three explanations consistent with this handout.